

Experiment 4: Continuous Integration / Continuous Development with Jenkins and Java & Cloning in GitHub

Aim

To implement Continuous Integration using Jenkins, GitHub, and a Java project.

Step 1: Install Java

Check Java:

```
java -version
```

If Java is not installed, install **JDK 21**.

Verify:

```
java -version
```

```
javac -version
```

Step 2: Install Jenkins

1. Download Jenkins MSI installer.
2. Run installer.
3. Select:

Run service as LocalSystem

4. When Jenkins asks for Java directory, provide:

C:\Program Files\Eclipse Adoptium\jdk-21.0.11.10-hotspot

(Not the bin folder)

5. Finish installation.
-

Step 3: Open Jenkins

Open browser:

<http://localhost:8080>

Unlock Jenkins using the initial admin password.

Install:

Install Suggested Plugins

Create Admin User.

Step 4: Create Java Project

Create folder:

```
mkdir JenkinsDemo  
cd JenkinsDemo
```

Create file:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello Jenkins");  
        System.out.println("This is a demo for Jenkins");  
    }  
}
```

Test:

```
javac Hello.java  
java Hello
```

Output:

```
Hello Jenkins  
This is a demo for Jenkins
```

Step 5: Push Project to GitHub

Initialize repository:

```
git init
```

Add files:

```
git add .
```

Commit:

```
git commit -m "Initial Commit"
```

Create GitHub repository.

Connect repository:

```
git remote add origin https://github.com/username/Java24.git
```

Push:

git branch -M main

git push -u origin main

Step 6: Create Jenkins Job

In Jenkins:

New Item

Enter:

Java-CI

Select:

Freestyle Project

Click:

OK

Step 7: Configure GitHub Repository

Source Code Management:

Git

Repository URL:

<https://github.com/username/Java24.git>

Branch:

*/main

 Important:

Use:

*/main

not

*/master

otherwise build fails.

Step 8: Configure Build Step

Build → Add Build Step

Select:

Execute Windows batch command

Command:

```
cd JenkinsDemo
javac Hello.java
java Hello
```

Step 9: Build Project

Click:

Build Now

Open:

Build History

Click build number.

Open:

Console Output

Expected:

```
Hello Jenkins
This is a demo for Jenkins
```

Finished: SUCCESS

Architecture Diagram for Viva

Developer



GitHub Repository



Jenkins



Clone Repository



Compile Java Program

↓

Execute Program

↓

SUCCESS

Viva Questions

What is Jenkins?

Jenkins is an open-source automation server used for Continuous Integration and Continuous Delivery (CI/CD).

What is Continuous Integration?

Continuous Integration is the process of automatically building and testing code whenever changes are pushed to a repository.

What is GitHub?

GitHub is a cloud-based platform used for storing and managing Git repositories.

What is Cloning?

Cloning means creating a local copy of a remote repository.

```
git clone repository_url
```

What is a Jenkins Job?

A Jenkins Job is a configured task that performs operations such as build, test, or deployment.

What was the issue you faced?

Jenkins was looking for:

master

branch while the repository contained:

main

branch.

Fixed by changing:

*/master

to

*/main

How do you know build succeeded?

Console Output shows:

Finished: SUCCESS

30-Second Viva Answer

"I created a Java project, pushed it to GitHub, installed Jenkins, created a Freestyle Project, connected Jenkins with the GitHub repository, configured a build step using javac and java, and executed the build. Jenkins cloned the repository, compiled the Java code, executed it, and displayed Finished: SUCCESS, demonstrating Continuous Integration."

This summary is enough to reproduce the entire experiment in the lab within **10–15 minutes** and answer most viva questions confidently. 🚀

The examiner may ask:

"Where exactly did Continuous Integration happen in your experiment?"

Many students say "Jenkins and GitHub are connected" but cannot prove the CI part.

What is Continuous Integration?

Continuous Integration means:

Developer changes code

↓

Pushes code to GitHub

↓

Jenkins fetches latest code

↓

Builds/Tests code automatically

Where did YOU perform CI?

Step 1: You modified the code

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello Jenkins");  
        System.out.println("This is a demo for Jenkins");  
    }  
}
```

Step 2: You committed and pushed to GitHub

```
git add .  
git commit -m "Jenkins Project part 2"  
git push origin main
```

This created the commit:

Commit message: "Jenkins Project part 2"

Step 3: Jenkins fetched the latest code from GitHub

From your console output:

Fetching upstream changes from
<https://github.com/shreyash-malani/Java24.git>

This proves Jenkins connected to GitHub and downloaded the latest code.

Step 4: Jenkins checked out the latest commit

Your console shows:

Checking out Revision
06e53959840a37b355ac057117bfefd3b12ad9f3

This proves Jenkins fetched the latest version from GitHub.

Step 5: Jenkins built the code

Console output:

```
javac Hello.java
```

Jenkins compiled your Java program.

Step 6: Jenkins executed the code

Console output:

```
java Hello
```

Program execution happened through Jenkins.

Step 7: Build succeeded

Console output:

Finished: SUCCESS

This confirms successful integration and build.

Viva Answer

"Continuous Integration was performed when I pushed my Java project to GitHub and Jenkins automatically fetched the latest commit, checked out the code, compiled it using javac, executed it using java, and generated a successful build. The Jenkins console output showing repository fetch, checkout, build, and Finished: SUCCESS proves that CI was implemented."

One Important Point

Technically, in a real CI pipeline, Jenkins should trigger the build **automatically** after a GitHub push using a webhook.

In your experiment, you clicked **Build Now** manually.

So if the examiner asks:

"Was the build automatic?"

Answer:

"No sir, in this experiment I manually triggered the Jenkins build using 'Build Now'. However, Jenkins successfully integrated with GitHub, fetched the latest code, compiled it, and executed it. Automatic triggering can be achieved using GitHub Webhooks."

This answer is technically correct and usually impresses examiners because it shows you understand the difference between **manual build** and **fully automated CI**.